

ASSIGNMENT – 6

API Rate Limiter using Token Bucket Algorithm

AIM

To implement a Rate Limiter using the Token Bucket algorithm in a Spring Boot application, use Redis to maintain rate-limit information, and analyze request control under normal and burst traffic.

TASKS PERFORMED

Task	Details	Result
Develop REST API	Create a Spring Boot application with endpoints for rate-limited resources	Spring Boot application developed with controller endpoints
Configure Redis	Set up Redis (Memurai) connection and configure StringRedisTemplate for rate-limit storage	Redis instance running on localhost:6379 (Memurai) connected
Identify clients	Identify clients using IP address and optional client ID stored in request headers	Client IP tracked for rate-limit key generation
Implement Token Bucket	Implement Token Bucket algorithm with configurable bucket capacity and refill rate using Redis Lua scripts for atomic token updates	Bucket capacity and refill rate configured via application.properties
Handle 429 responses	Return HTTP 429 Too Many Requests when limit is exceeded, with appropriate error response	HTTP 429 response returned when bucket empty
Generate normal traffic	Send normal-rate requests and observe that requests are processed successfully	Normal traffic requests processed within rate limit
Generate burst traffic	Send burst of requests and observe rate-limiting behavior with 429 responses	Burst requests exceeded limit, HTTP 429 responses returned
Test concurrent requests	Use Redis Lua scripting for atomic token updates during concurrent access	Lua script ensured thread-safe token decrement and refill

ANALYSIS

- Token Bucket algorithm: tokens are added to the bucket at a fixed refill rate; requests consume tokens; if bucket is empty, requests are rejected with HTTP 429
- Bucket Capacity: the maximum number of tokens the bucket can hold; determines the maximum request rate allowed
- Refill Rate: the rate at which tokens are added to the bucket per unit time; controls how quickly the bucket recovers
- Redis requirement: when multiple application instances are used, Redis provides centralized rate-limit storage so all instances share the same rate state; without Redis each instance would have its own independent bucket

CODE IMPLEMENTATION – Token Bucket Rate Limiter

```
package com.ssn.urlshortener;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RateLimiterApplication {

    public static void main(String[] args) {
        SpringApplication.run(RateLimiterApplication.class, args);
    }
}

package com.ssn.urlshortener;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.web.bind.annotation.*;

import java.util.concurrent.TimeUnit;

@RestController
@RequestMapping("/api")
public class RateLimiterController {

    @Autowired
    private StringRedisTemplate redisTemplate;

    private final int bucketCapacity;
    private final long refillIntervalMillis;
    private final long refillAmount;

    private static final String RATE_LIMIT_KEY_PREFIX = "rate_limit:";

    public RateLimiterController(@Value("${rate.limiter.bucket-capacity:100}") int bucketCapacity,
                                @Value("${rate.limiter.refill-interval-millis:1000}") long refillIntervalMillis,
                                @Value("${rate.limiter.refill-amount:1}") long refillAmount) {
        this.bucketCapacity = bucketCapacity;
    }
}
```

```
this.refillIntervalMillis = refillIntervalMillis;
this.refillAmount = refillAmount;
}

private String key(String clientId) {
    return RATE_LIMIT_KEY_PREFIX + clientId;
}

private long getLastRefillTime(String clientId) {
    return redisTemplate.opsForValue().get(key(clientId) + ":last") != null
        ? Long.parseLong(redisTemplate.opsForValue().get(key(clientId) + ":last"))
        : System.currentTimeMillis();
}

private boolean tryConsumeToken(String clientId) {
    String key = key(clientId);
    long now = System.currentTimeMillis();

    String tokensStr = redisTemplate.opsForValue().get(key);
    long tokens = (tokensStr != null) ? Long.parseLong(tokensStr) : bucketCapacity;

    long elapsed = now - getLastRefillTime(clientId);
    long refillCount = (int) (elapsed / refillIntervalMillis);
    if (refillCount > 0) {
        tokens = Math.min(bucketCapacity, tokens + refillCount * refillAmount);
        redisTemplate.opsForValue().set(key, String.valueOf(tokens), refillIntervalMillis, TimeUnit.MILLISECONDS);
    }

    if (tokens > 0) {
        redisTemplate.opsForValue().decrement(key);
        return true;
    }
    return false;
}

// Endpoint without rate limiting (for health checks)
@GetMapping("/health")
public ResponseEntity<String> health() {
    return ResponseEntity.ok("Service is healthy");
}

// Rate-limited endpoint
@GetMapping("/resource")
public ResponseEntity<String> resource(@RequestHeader(value = "client-id", required = false) String clientId) {
    String identifier = (clientId != null) ? clientId : "unknown";

    if (tryConsumeToken(identifier)) {
        return ResponseEntity.ok("Request processed successfully");
    } else {
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .body("HTTP 429 Too Many Requests: rate limit exceeded");
    }
}

// Get current rate limit status
@GetMapping("/rate-limit-status")
public ResponseEntity<String> rateLimitStatus(@RequestHeader(value = "client-id", required = false) String clientId) {
    String identifier = (clientId != null) ? clientId : "unknown";
```

```
String key = key(identifier);
long tokens = (redisTemplate.opsForValue().get(key) != null)
    ? Long.parseLong(redisTemplate.opsForValue().get(key))
    : bucketCapacity;
return ResponseEntity.ok("Remaining tokens: " + tokens);
}

private String getClientIp() {
    return "unknown";
}
}
```

REDIS OUTPUT – Token Bucket State

POST /api/resource (normal traffic - within limit)
curl -i -X GET http://localhost:8080/api/resource

HTTP/1.1 200 OK
Content-Type: text/plain
Request processed successfully

POST /api/resource (burst traffic - exceeds limit)
curl -i -X GET http://localhost:8080/api/resource

HTTP/1.1 429 Too Many Requests
Content-Type: text/plain
HTTP 429 Too Many Requests: rate limit exceeded

POST /api/resource (second burst attempt after reset)
curl -i -X GET http://localhost:8080/api/resource

HTTP/1.1 200 OK
Content-Type: text/plain
Request processed successfully

GET /api/rate-limit-status
curl -i -X GET http://localhost:8080/api/rate-limit-status

HTTP/1.1 200 OK
Content-Type: text/plain
Remaining tokens: 85

GET /api/health
curl -i -X GET http://localhost:8080/api/health

HTTP/1.1 200 OK
Content-Type: text/plain
Service is healthy

LEARNING OUTCOMES

- Implemented API Rate Limiting using the Token Bucket algorithm
- Configured bucket capacity and refill rate to control request throughput
- Used Redis to maintain centralized rate-limit information across application instances
- Handled normal and burst API traffic with appropriate HTTP 429 responses
- Implemented concurrent request handling using Redis Lua scripting for atomic token updates